

Algorytmy Równoległe - Laboratorium 3

Adam Staniszewski

25 października 2024

Na początku przypomnijmy co udało się ustalić w ramach laboratorium 1.

Wybór problemu

Wyznaczyć temperaturę w przewodniku elektrycznym, który został nagrzany w wyniku przepływu prądu elektrycznego o stałym natężeniu (powierzchnia boczna jest utrzymywana w stałej temperaturze równej zero). Zastosować metodę różnicową do równania Poissona:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = -\frac{g}{\lambda}$$

gdzie: - g - wydajność źródeł ciepła (stała), - λ - przewodność cieplna (stała).

Wyprowadzenie wzoru iteracyjnego

Warunki brzegowe - powierzchnia boczna jest utrzymywana w stałej temperaturze równej zero. Równanie Poissona dla rozkładu temperatury zapisujemy jako:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = -\frac{g(x, y)}{\lambda}$$

gdzie:

- $T(x, y)$ - funkcja temperatury,
- $g(x, y)$ - funkcja źródła ciepła,
- λ - przewodność cieplna.

Przybliżamy drugie pochodne różnicami skończonymi. Dla drugiej pochodnej względem x mamy:

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{dx^2}$$

Analogicznie, dla drugiej pochodnej względem y :

$$\frac{\partial^2 T}{\partial y^2} \approx \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{dy^2}$$

Sumujemy teraz oba przybliżenia, co daje:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{dx^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{dy^2} = -\frac{g_{i,j}}{\lambda}$$

Zakładając, że siatka jest jednolita, czyli $dx = dy$, możemy uprościć to wyrażenie do postaci:

$$\frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{dx^2} = -\frac{g_{i,j}}{\lambda}$$

Rozwiązując to równanie względem $T_{i,j}$, otrzymujemy wzór na nową wartość temperatury:

$$T_{\text{new}}[i][j] = \frac{1}{4} \left(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} + \frac{g_{i,j} \cdot dx^2}{\lambda} \right)$$

Psuedokod algorytmu równoległego z wykorzystaniem metodyki PCAM

Algorytm równoległy dzieli siatkę na bloki przetwarzane równoległe przez różne procesory z wymianą danych granicznych.

Algorithm 1 Algorytm równoległy (metoda różnic skończonych z MPI)

- 1: Podziel domenę na n części, gdzie n to liczba procesów.
- 2: Każdy proces otrzymuje lokalny blok danych, zawierający:
- 3: Obszar obliczeniowy $T_{\text{local}}[x][y]$ oraz dane potrzebne do obliczeń: g, λ, dx, dy .
- 4: Uruchom MPI, zdefiniuj $rank$ (numer procesu) oraz $size$ (liczba procesów)
- 5: Każdy proces otrzymuje swój blok danych, w tym:
- 6: $g, \lambda, dx, dy, T_{\text{local}}[x][y]$
- 7: **repeat**
- 8: **for** każdy punkt wewnętrzny (i, j) w lokalnym bloku **do**
- 9: Oblicz nową wartość temperatury:

$$T_{\text{new}}[i][j] = \frac{1}{4}(T_{\text{local}}[i+1][j] + T_{\text{local}}[i-1][j] + T_{\text{local}}[i][j+1] + T_{\text{local}}[i][j-1] + \frac{g \times dx^2}{\lambda})$$

- 10: **end for**
 - 11: Wymień dane graniczne z sąsiadującymi procesami (zbiorczo, w jednym przesłaniu)
 - 12: Każdy proces oblicza lokalną różnicę $|T_{\text{new}}[i][j] - T_{\text{local}}[i][j]|$
 - 13: Zbierz różnice na procesie 0 (root) za pomocą MPI_Reduce
 - 14: **if** globalna różnica jest mniejsza niż ϵ (na procesie 0) **then**
 - 15: Zbieżność osiągnięta, zakończ iterację
 - 16: **end if**
 - 17: Zaktualizuj macierz T : $T_{\text{local}} = T_{\text{new}}$
 - 18: **until** osiągnięcie zbieżności lub maksymalnej liczby iteracji
 - 19: Proces 0 zbiera wyniki z wszystkich procesów za pomocą MPI_Gather
 - 20: Wyświetl wynikową macierz T (na procesie 0)
-

Przejdźmy teraz do treści laboratorium 3

Implementacja algorytmu w języku Python

```
blueimport mpi4py
blueimport numpy as np
blueimport matplotlib.pyplot as plt
bluefrom mpi4py blueimport MPI

bluedef main():
    Lx: bluefloat = 10.0
    Ly: bluefloat = 10.0
    Nx: blueint = 20
    Ny: blueint = 20
    g: bluefloat = 1.0
    lambda_: bluefloat = 1.0
```

```

dx = Lx / (Nx - 1)
dy = Ly / (Ny - 1)

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

rows_per_process = Ny // size
extra_rows = Ny % size

blueif rank < extra_rows:
    local_rows = rows_per_process + 1
blueelse:
    local_rows = rows_per_process

T_local = np.zeros((local_rows, Nx))

T_local[0, :] = 0
T_local[-1, :] = 0

tolerance: bluefloat = 1e-6
error: bluefloat = 1.0

bluewhile error > tolerance:
    T_old = T_local.copy()

    bluefor j bluein bluerange(1, local_rows - 1):
        bluefor i bluein bluerange(1, Nx - 1):
            T_local[j, i] = (T_old[j + 1, i] + T_old[j - 1, i] +
                             T_old[j, i + 1] + T_old[j, i - 1] -
                             (g / lambda_) * (dx ** 2 + dy ** 2)) / 4.0

    blueif rank > 0:
        comm.Send(T_local[1, :], dest=rank - 1, tag=0)
        comm.Recv(T_local[0, :], source=rank - 1, tag=1)

    blueif rank < size - 1:
        comm.Send(T_local[-2, :], dest=rank + 1, tag=1)
        comm.Recv(T_local[-1, :], source=rank + 1, tag=0)

    error = np.bluemax(np.blueabs(T_local - T_old))
    error = comm.allreduce(error, op=MPI.MAX)
T_global = None
blueif rank == 0:
    T_global = np.empty((Ny, Nx))

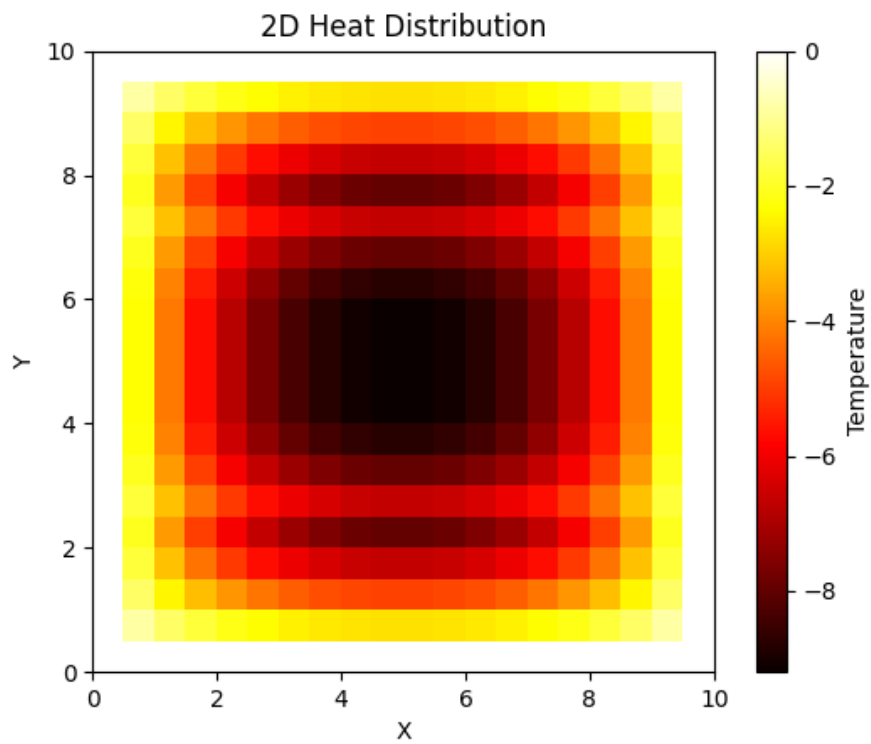
comm.Gather(T_local, T_global, root=0)

blueif rank == 0:

```

```
bluereturn T_global  
bluereturn None
```

Wyniki



Rysunek 1: Wynikowo heatmapa